

JS Async

[JS Callbacks](#)

[JS Asynchronous](#)

[JS Promises](#)

[JS Async/Await](#)



🔥 Part 1: JavaScript Asynchronous ka matlab kya hota hai ?

◆ JavaScript normal kaise kaam karta hai?

- JavaScript ek **line-by-line chalne wali language** hai.
- Pehle ek kaam hoga, phir doosra. Jab tak pehla complete nahi hota, doosra start nahi hota.

☐ Isse bolte hain **Synchronous**.

main.js	Output
<pre>1- function boilWater() { 2 console.log("1. Paani garam kar raha hoon 🔥"); 3 } 4- function addTeaLeaves() { 5 console.log("2. Chai patti daali 🍃"); 6 } 7- function addMilk() { 8 console.log("3. Doodh daala 🥛"); 9 } 10- function serveTea() { 11 console.log("4. Chai ready, serve kar di 🍵"); 12 } 13 14 // Line-by-line kaam ho raha hai (Synchronous) 15 boilWater(); 16 addTeaLeaves(); 17 addMilk(); 18 serveTea(); 19</pre>	<pre>1. Paani garam kar raha hoon 🔥 2. Chai patti daali 🍃 3. Doodh daala 🥛 4. Chai ready, serve kar di 🍵 === Code Execution Successful ===</pre>

Code -> [Click me](#)

🔥 Part 2: Asynchronous ka matlab ?

- Kuch kaam aise hote hain jo **turant nahi hote** (like: API se data lana, 5 sec wait karna).
- Ye kaam **background mein chal jaate hain**, aur JavaScript **aage badh jaata hai**.

☐ Isse bolte hain **Asynchronous**.

main.js	Output
<pre>1- function boilWater() { 2- setTimeout(() => { 3- console.log("1. Paani garam ho gaya 🔥"); 4- }, 2000); // 2 second ka wait 5- } 6 7- function addTealeaves() { 8- console.log("2. Chai patti daali 🍃"); 9- } 10- function addMilk() { 11- console.log("3. Doodh daala 🥛"); 12- } 13 14- function serveTea() { 15- console.log("4. Chai serve kar di 🍵"); 16- } 17 //-----> // Asynchronous execution <----- 18 boilWater(); // Ye background me chalu ho gaya 19 addTealeaves(); // Ye turant chala 20 addMilk(); // Ye bhi turant chala 21 serveTea(); // Ye bhi turant chala 22</pre>	<pre>2. Chai patti daali 🍃 3. Doodh daala 🥛 4. Chai serve kar di 🍵 1. Paani garam ho gaya 🔥 === Code Execution Successful ===</pre>

Code → [Click me](#)

🔥 Part 3: Callback – ek chhota helper function

☐ **Callback kya hota hai?**

Jab hum kisi function ko bolte hain:

“Tu kaam khatam hone ke baad ye function chala dena”

Us function ko **callback** kehte hain.

Code → [Click me](#)

main.js	Output
<pre> 1- function doHomework(subject, callback) { 2 console.log(`Homework start kar raha hoon: \${subject}`); 3 4- setTimeout(() => { 5 console.log(`Homework complete kar diya: \${subject}`); 6 callback(); // Jab homework complete ho jaye, ye chalega 7 }, 2000); 8 } 9 10- function takeBreak() { 11 console.log("Chalo break lete hain 🍷"); 12 } 13 14 // Function ko call karo aur callback pass karo 15 doHomework("Maths", takeBreak); 16 </pre>	<pre> Homework start kar raha hoon: Maths Homework complete kar diya: Maths Chalo break lete hain 🍷 === Code Execution Successful === </pre>

Ex-2

main.js	Output
<pre> 1- function orderFood(food, callback) { 2 console.log(`Order diya: \${food}`); 3 4- setTimeout(() => { 5 console.log(`\${food} ready ho gaya! 🍕`); 6 callback(); 7 }, 3000); 8 } 9 10- function eatFood() { 11 console.log("Ab khana khaate hain 😋"); 12 } 13 14 // Order karo aur callback pass karo 15 orderFood("Pizza", eatFood); 16 </pre>	<pre> Order diya: Pizza </pre>

After 2 Sec

Share	Run	Output
		<pre> Order diya: Pizza Pizza ready ho gaya! 🍕 Ab khana khaate hain 😋 === Code Execution Successful === </pre>

🔥 Part 4: Promises – Future ka promise ->

(async / await – Easy tarika Promises ka)

◆ English:

`async` and `await` are modern JavaScript keywords used to handle Promises in a simpler and cleaner way. They make asynchronous code look like synchronous code.

📖 Definition ():

`async` / `await` is a simpler and cleaner way to write asynchronous code using Promises. It makes your code look like it runs step-by-step (like normal code), but it still handles things that take time — like API calls, timers, etc.

Keyword	Simple Meaning
<code>async</code>	Use this before a function to make it return a Promise
<code>await</code>	Use this to wait until a Promise is done (resolved or rejected)
<code>try / catch</code>	Used to handle errors if the Promise is rejected

❑ Promise kya hai?

JavaScript bolta hai:

“Main tumko ek promise deta hoon. Agar kaam sahi se ho gaya, to success (resolve) bolunga. Nahi hua to error (reject)”

➔(PROMISE)

◆ Ye bataane ke liye ki kaam abhi chal raha hai, future me hoga complete.

"Promise" ka matlab hota hai **“Main future me kuch kar dunga”**
Isliye hum kehte hain:

◆ **“Khaana ban raha hai”** (abhi bana nahi, process me hai)

◆ **“Result aane wala hai”**

◆ **“Kaam ho raha hai”**

JavaScript me bhi same concept hai.

🔗 *Jab hum new Promise() banate hain:*

- Hum JavaScript ko bol rahe hote hain:
“Yeh kaam abhi chal raha hai, aur baad me batayenge kya hua (success ya error)”

```
JS code_1.js > ...
1 let meraPromise = new Promise(function(resolve, reject) {
2   let kaamHoGaya = true;
3
4   if (kaamHoGaya) {
5     resolve("✅ Kaam sahi se ho gaya! Success!");
6   } else {
7     reject("❌ Arre bhai, kaam nahi hua. Error!");
8   }
9 };
10
11 meraPromise
12 .then(function(successMessage) {
13   console.log(successMessage);
14 })
15 .catch(function(errorMessage) {
16   console.log(errorMessage);
17 });
18
```

Click me → [click](#)

🔑 Keyword	📖 Matlab / Kaam
<code>let</code>	Variable banata hai. Yahan <code>meraPromise</code> naam ka variable banaya.
<code>new</code>	Naya object banane ke liye hota hai. Yahan <code>new Promise</code> banaya.
<code>Promise</code>	JavaScript ka object jo future me kaam hone ka wada karta hai.
<code>function</code>	Ek function banata hai. Yahan anonymous function use hua hai.
<code>resolve</code>	Jab kaam sahi ho jaye to <code>resolve()</code> call hota hai.
<code>reject</code>	Jab kaam fail ho jaye to <code>reject()</code> call hota hai.

```
let kaamHoGaya =  
    true
```

```
if (kaamHoGaya)
```

```
    .then()
```

```
    .catch()
```

```
    console.log()
```

```
    successMessage
```

```
    errorMessage
```

Ek condition hai. Agar kaam hua toh `true`, warna `false`.

Condition check karta hai – agar `true` hai to `resolve`, warna `reject`.

Jab Promise **safalta se pura hota hai**, to `.then()` chalega.

Jab Promise **fail hota hai**, to `.catch()` chalega.

Output ko screen pe dikhane ke liye use hota hai.

`resolve()` se jo message aaya, wo is variable me milta hai.

`reject()` se jo message aaya, wo isme milta hai.

Ex-2

```
JS code_3.js > [?] khaanaBanRahaHai > <function>  
1  let khaanaBanRahaHai = new Promise(function(resolve, reject) {  
2      let gasHai = false;  
3  
4      if (gasHai) {  
5          resolve("✅ Mummy ne khaana bana diya! Aaja khane!");  
6      } else {  
7          reject("❌ Gas nahi thi, khaana nahi bana...");  
8      }  
9  });  
10  
11  khaanaBanRahaHai  
12  .then(function(msg) {  
13      console.log(msg); // Jab kaam ho jaye  
14  })  
15  .catch(function(err) {  
16      console.log(err); // Jab kaam fail ho jaye  
17  });  
18
```

Click → [Click me](#)

Ex-3

```
JS code_2.js > <function>
1
2  function checkResult(marks) {
3    return new Promise(function(resolve, reject) {
4      console.log("Result check ho raha hai...");
5
6      setTimeout(() => {
7        if (marks >= 35) {
8          resolve("✅ Pass ho gaya balleeee ballelee ! Marks: " + marks);
9        } else {
10         reject("❌ Fail ho gaya ro rh hu bhaiiii main to .... ! : " + marks);
11       }
12     }, 2000);
13   });
14 }
15
16 let studentMarks = 2;
17
18 checkResult(studentMarks)
19   .then(function(successMsg) {
20     console.log(successMsg);
21   })
22   .catch(function(errorMsg) {
23     console.log(errorMsg);
24   });
25
```

Code → [click me](#)



Short Overview: Promise & async/await in JavaScript

◆ What is a Promise?

A **Promise** is a way to handle tasks that take time (asynchronous tasks).
It gives a **result in the future** — either:

- ✅ `resolve()` → if the task is successful
- ❌ `reject()` → if the task fails

🔥 Asynchronous Tasks kya hote hain ?

Asynchronous = “alag se chalne wala kaam” ya “background mein chalne wala kaam”

Example samajh:

- Jab tum mobile me game khel raha ho,
- Aur saath hi **music chal raha ho**,
- Ya phone pe **message aata ho**,

Toh game chal raha hai, aur background me message ya music bhi chal raha hai — dono ek saath bina rukawat ke.

Type	Matlab	Kaam Kaise Hota Hai?
Synchronous	Line by line, ek ke baad ek	Ek kaam pura tabhi doosra shuru hota hai
Asynchronous	Background me chalne wala kaam	Kaam ek saath ya background me hota hai